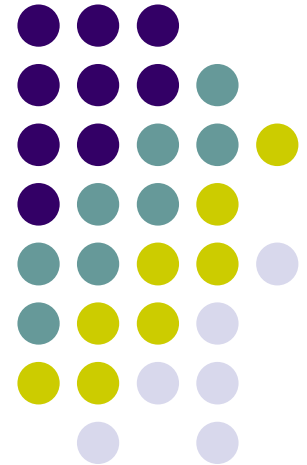
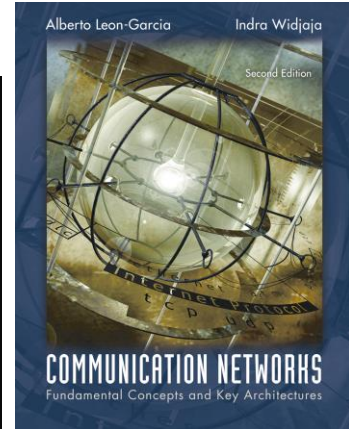


Chapter 2

Applications and Layered Architectures

Protocols, Services & Layering
OSI Reference Model
TCP/IP Architecture
How the Layers Work Together



Layers, Services & Protocols



- The overall communications process between two or more machines connected across one or more networks is very complex
- ***Layering*** partitions related communications functions into groups that are manageable
- Each layer provides a ***service*** to the layer above
- Each layer operates according to a ***protocol***

Why Layering?



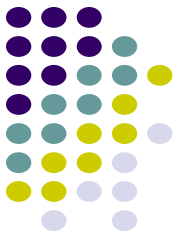
- Layering simplifies design, implementation, and testing by partitioning overall communications process into parts
- Protocol in each layer can be designed separately from those in other layers
- Protocol makes “calls” for services from layer below
- Layering provides flexibility for modifying and evolving protocols and services without having to change layers below
- Non-layered architectures are costly, inflexible, and soon become outdated

Protocols



- A *protocol* is a set of rules that governs how two or more communicating entities in a layer are to interact
- *Messages* that can be sent and received
- *Actions* that are to be taken when a certain event occurs, e.g. sending or receiving messages, expiry of timers
- **The purpose of a protocol is to provide a service to the layer above**

Examples of Layers



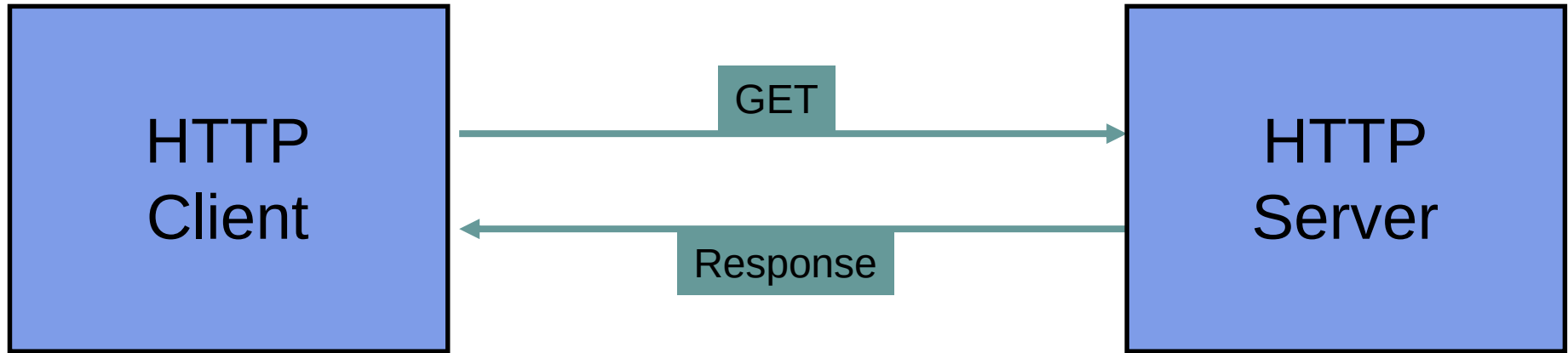
- Application Layer: communication functions that are used by application programs
 - HTTP (web browsing), SMTP (email)
- Transport Layer: end-to-end communication between two processes in two machines
 - Transmission Control Protocol (TCP)
 - User Datagram Protocol (UDP)
- Network Layer: node-to-node communication between two machines
 - Internet Protocol (IP)



Example: HTTP

- HTTP is an application layer protocol
- Retrieves documents on behalf of a browser application program
- HTTP specifies fields in request messages and response messages
 - Request types; Response codes
 - Content type, options, cookies, ...
- HTTP specifies actions to be taken upon receipt of certain messages

HTTP Protocol



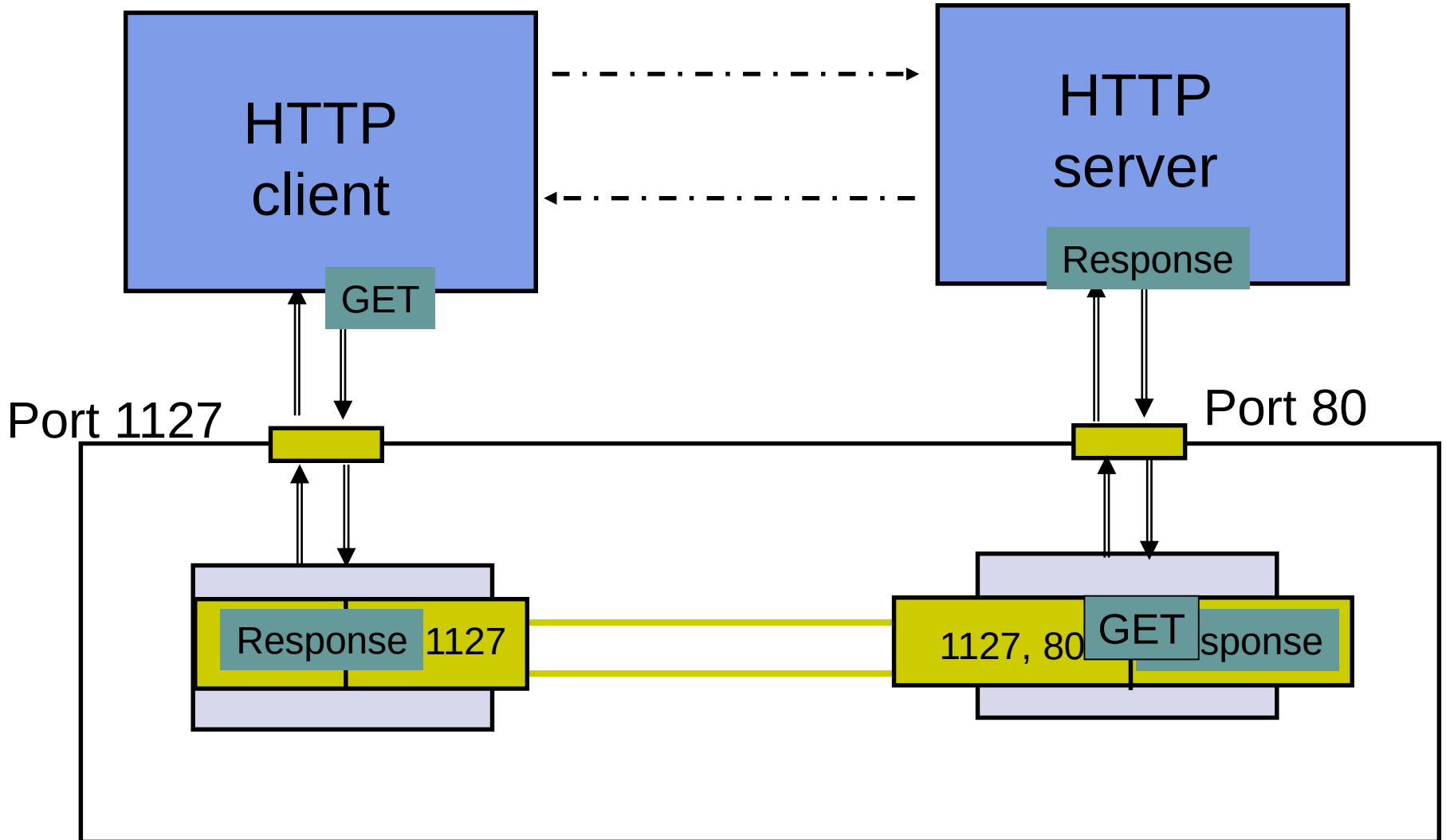
- HTTP assumes messages can be exchanged directly between HTTP client and HTTP server
- In fact, HTTP client and server are processes running in two different machines across the Internet
- HTTP uses the reliable stream transfer service provided by TCP

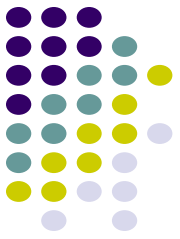
Example: TCP



- TCP is a transport layer protocol
- Provides *reliable byte stream service* between two processes in two computers across the Internet
- Sequence numbers keep track of the bytes that have been transmitted and received
- Error detection and retransmission used to recover from transmission errors and losses
- TCP is *connection-oriented*: the sender and receiver must first establish an association and set initial sequence numbers before data is transferred
- Connection ID is specified uniquely by
(*send port #, send IP address, receive port #, receiver IP address*)

HTTP uses service of TCP





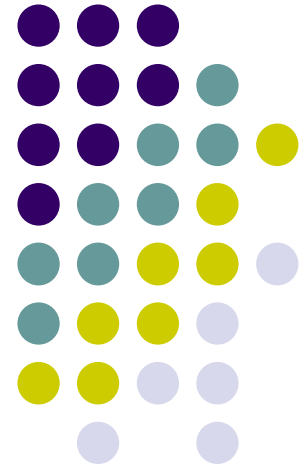
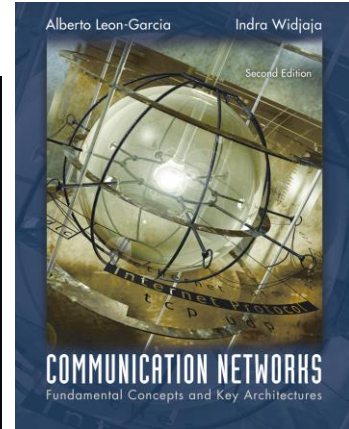
Summary

- Layers: related communications functions
 - Application Layer: HTTP, DNS
 - Transport Layer: TCP, UDP
 - Network Layer: IP
- Services: a protocol provides a communication service to the layer above
 - TCP provides connection-oriented reliable byte transfer service
 - UDP provides best-effort datagram service
- Each layer builds on services of lower layers
 - HTTP builds on TCP
 - DNS builds on top UDP
 - TCP and UDP build on IP

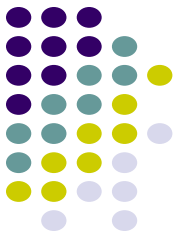
Chapter 2

Applications and Layered Architectures

OSI Reference Model



Open Systems Interconnection



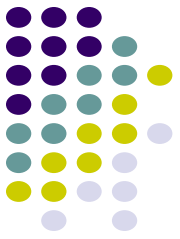
- Network architecture:
 - Definition of all the layers
 - Design of protocols for every layer
- By the 1970s every computer vendor had developed its own proprietary layered network architecture
- Problem: computers from different vendors could not be networked together
- Open Systems Interconnection (OSI) was an international effort by the International Organization for Standardization (ISO) to enable multivendor computer interconnection
- This is now a standard process in any new technology (e.g. WiMax, WLAN)

OSI Reference Model



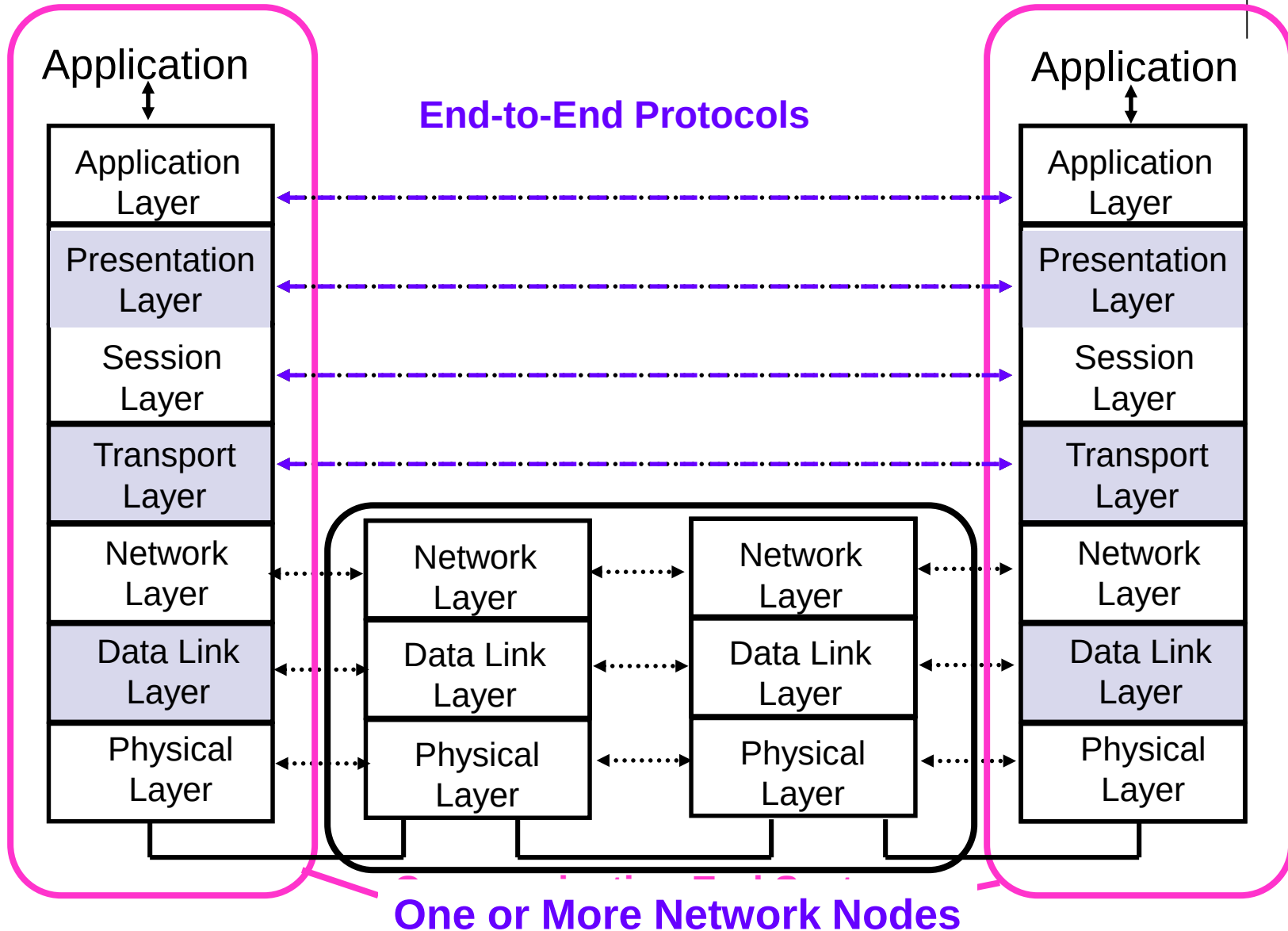
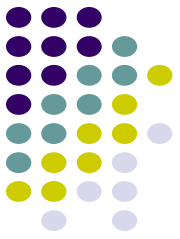
- Divides basic communication functions needed for two computers to communicate into 7 layers
- Describes a seven-layer abstract reference model for network architectures
- Purpose of the reference model was to provide a framework for the development of protocols

OSI Reference Model

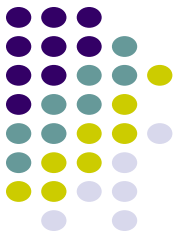


- OSI also provided a unified view of layers, protocols, and services which is still in use in the development of new protocols
- Detailed standards were developed for each layer, but most of these are not in use
- TCP/IP protocols preempted deployment of OSI protocols

7-Layer OSI Reference Model

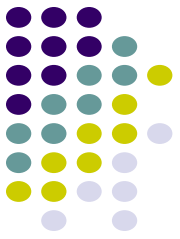


Physical Layer



- Transfers bits across link (units are bits)
- Deals with the transmission system and transmission media
 - DSL, cable modem, telephone modems...
 - Twisted-pair cable, coaxial cable optical fiber, radio, infrared, ...





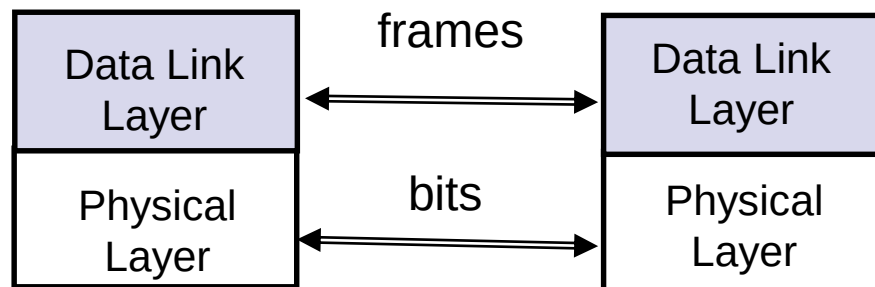
Physical Layer (2)

- Definition & specification of the physical aspects of a communications link
 - Mechanical: cable, plugs, pins...
 - Electrical/optical: modulation, signal strength, voltage levels, bit times, signal duration...
 - functional/procedural: how to activate, maintain, and deactivate physical links... (like a phone call)

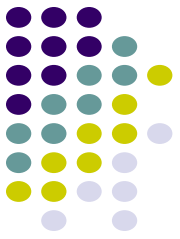
Data Link Layer



- Transfers *frames* (blocks of info) across *direct* connections
- Groups bits into frames; adds check bits
- Detection of bit errors; Retransmission of frames
- Activation, maintenance, & deactivation of data link connections

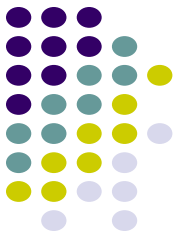


Data Link Layer



- Link includes case where multiple users need to be connected to the medium (example?)
- Medium access control for local area networks
- Flow control (so as not to overwhelm buffer on other end)

Network Layer



- Transfers *packets* across multiple links and/or multiple networks
(no pt-2-pt any more)
- Addressing must scale to large networks
- Nodes *jointly* execute routing algorithm to determine paths across the network
(makes net. layer most complex)

Network Layer (2)

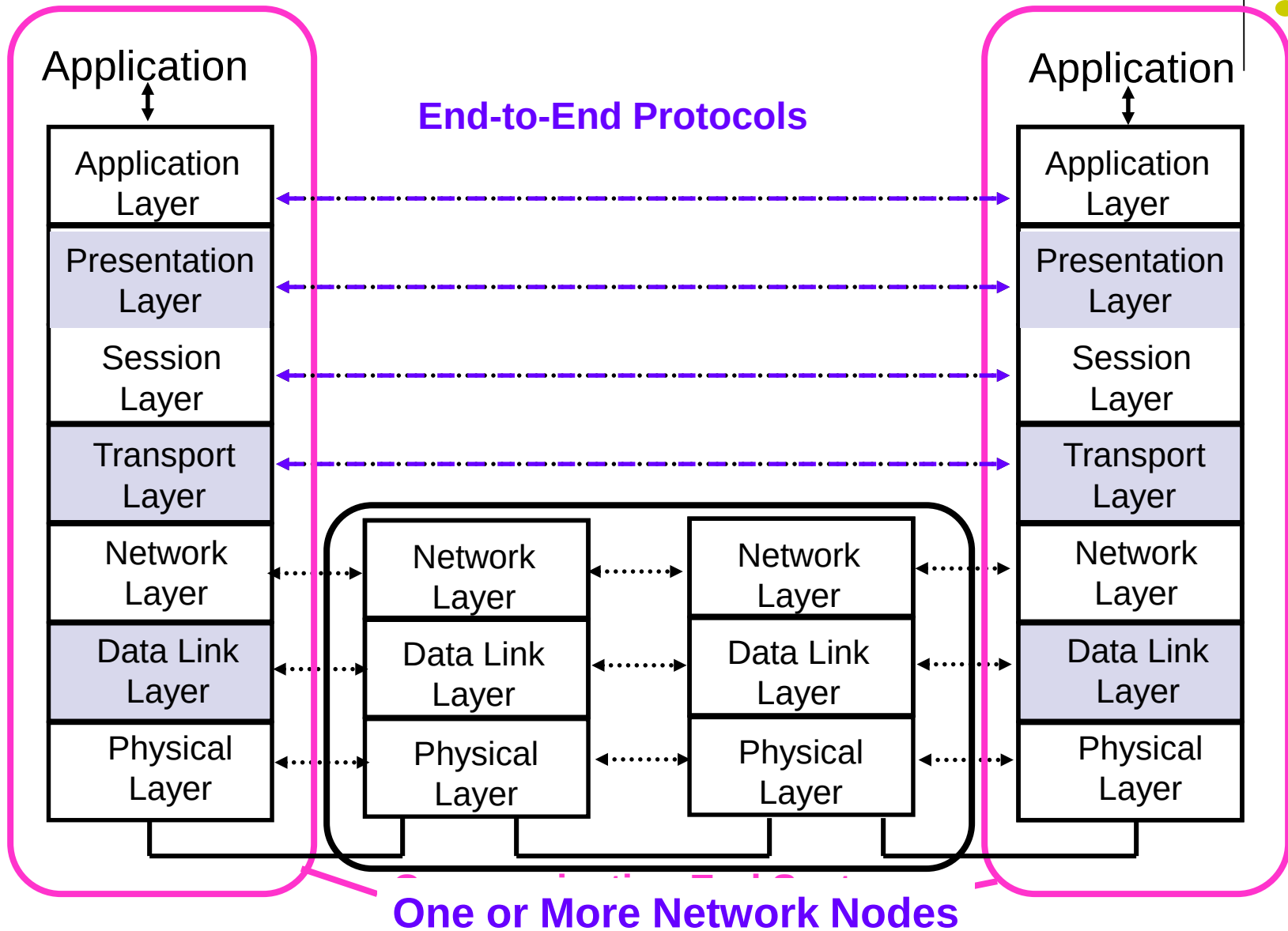


- Routing: procedure to select path across net
- When two machines are connected through
 - Same PS net: single address sp and routing procedure
 - Different nets:
 - nets differ in internal routing/addressing/packet size
 - Need an internetwork protocol

Network Layer (3)



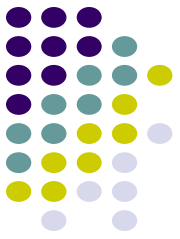
- Forwarding transfers packet across a node
- Congestion control to deal with traffic surges
- Connection setup, maintenance, and teardown when connection-based



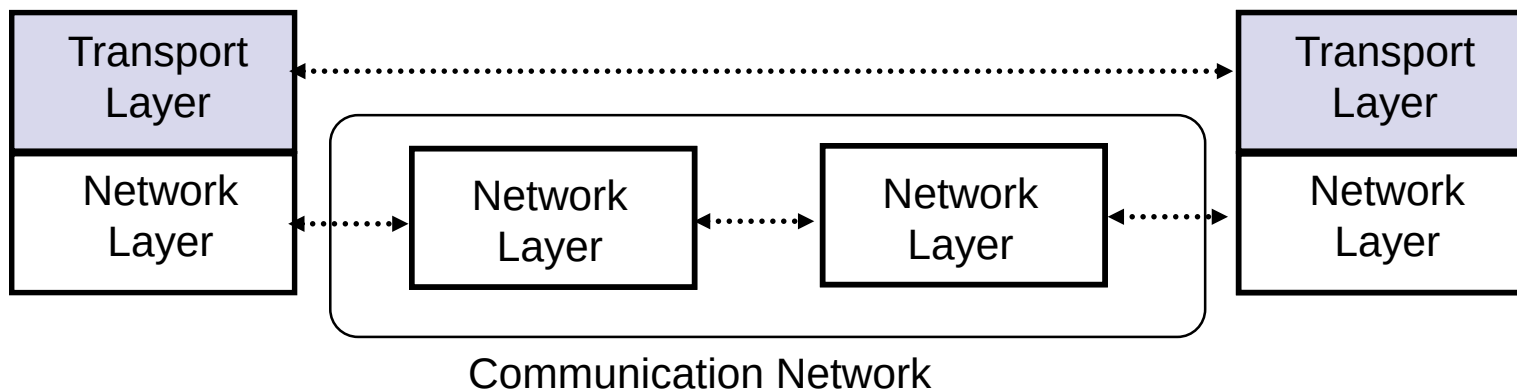


- Each node must implement lower 3 layers
- Lower 2 layers involve interaction of peer-2-peer processes across a single link
- Net. Layer in source and destination are not peers (**don't talk directly**)
- Nodes in Net. Layer jointly execute routing
- Top 4 layers involve interaction of peer processes across net.

Transport Layer



- Transfers data end-to-end from process in a machine to process in another machine:
 - Reliable stream transfer: error free transfer of a seq. of bytes or messages
 - or quick-and-simple (unreliable) single-block transfer (just provide appropriate address)





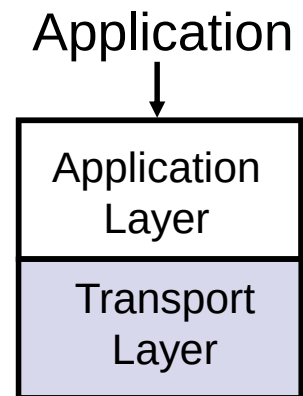
- Multiplexing: for opt. net service, multiplex several trans. layer connections into a single net. layer connection
- Multiplexing enabled by port numbers
- Splitting: for high throughput, layer can use splitting to support connection over several net layer connection
- Message segmentation and reassembly
- Connection setup, maintenance, and release

Application & Upper Layers



- Application Layer: Provides services that are frequently required by applications, like: web access, file transfer, email...
- Presentation Layer: machine-independent representation of data.. (different machines use different ways to represent integers and no.)
- Session Layer: dialog management (how data is exchanged, e.g. half or full duplex)

**Incorporated into
Application Layer**



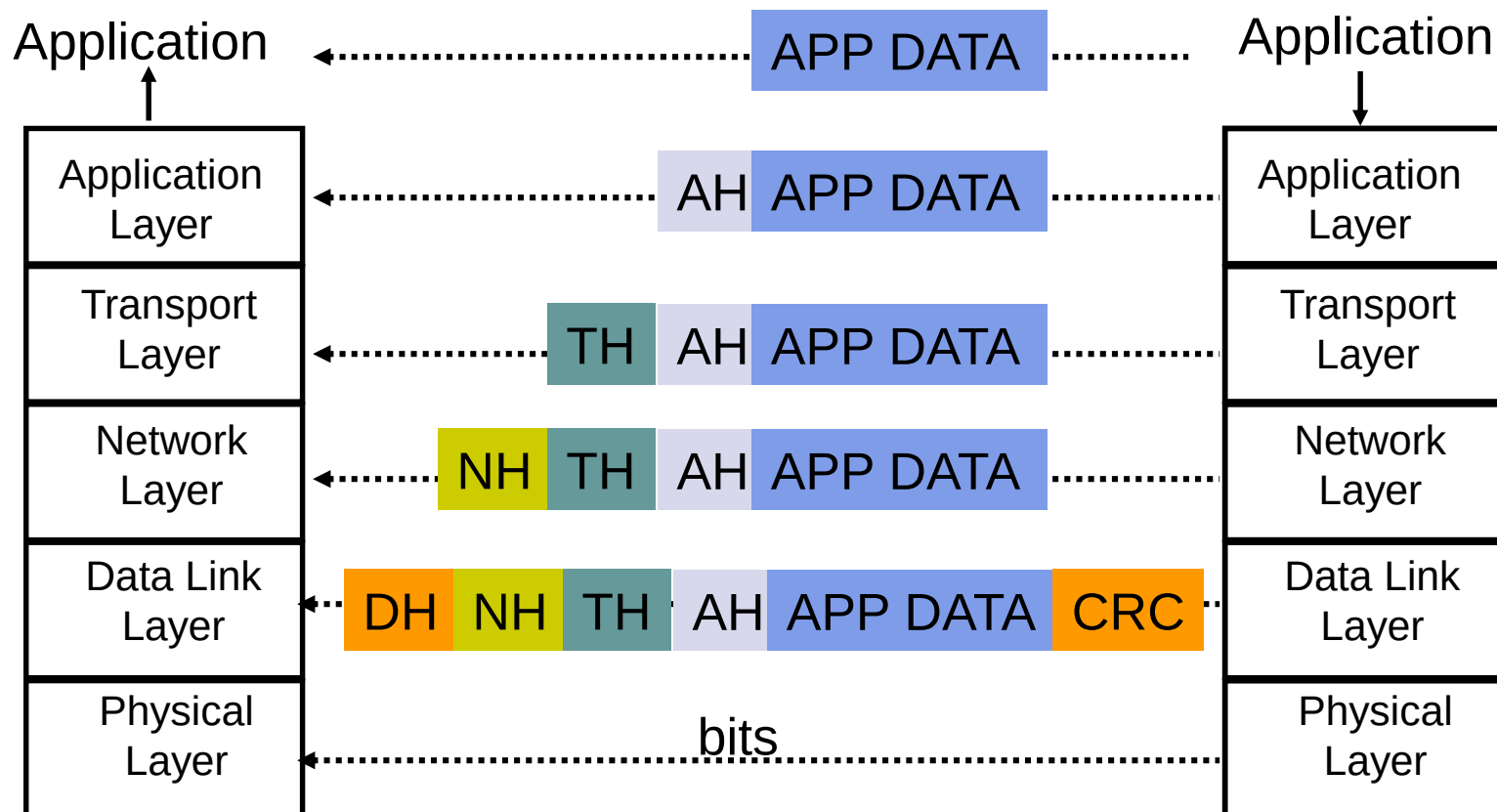


- OSI wanted to develop
- A layering model
- Standards for computer nets (protocols use)
- By time protocols were developed, TCP/IP emerged as an alternative (time to market)



Headers & Trailers

- Each protocol uses a header carrying control information: address, sequence number, flags, size indicators, etc...
- Check bits may be appended (trailer) for error detection



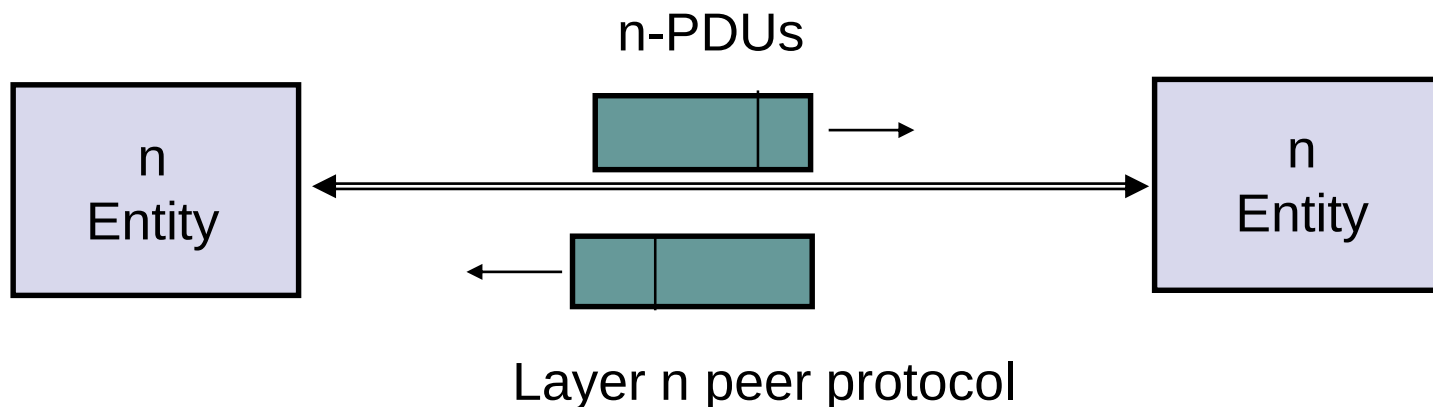


- At destination,
 1. each layer reads header and trailer info to determine action to be taken
 2. Strips header and trailer
 3. Passes forward to higher layer

OSI Unified View: Protocols



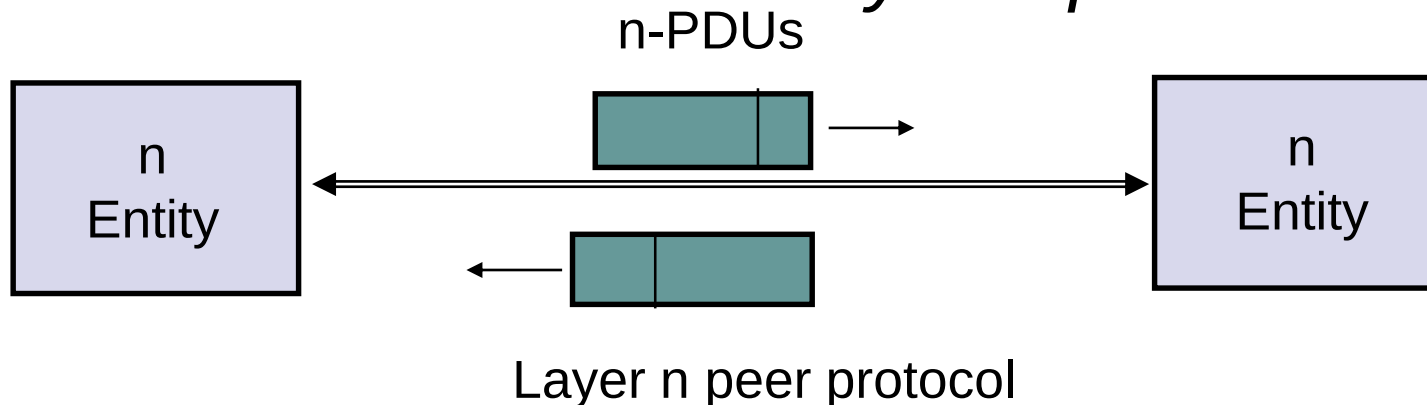
- The entities comprising the corresponding layers on different machines are called *peer processes*.
- A process on one machine interacts with a process on another machine across a peer interface (HTTP on client and server are peer processes)
- Layer-n peer processes communicate by exchanging *Protocol Data Units* (PDUs) (bits, frames, packets, segments)



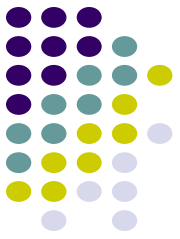
OSI Unified View: Protocols



- Each PDU contains a header and possibly a trailer
- Layer n in one machine interacts with layer n in another machine to provide a service to layer $n + 1$
- The machines use a set of rules and conventions called the *layer- n protocol*.



OSI Unified View: Services



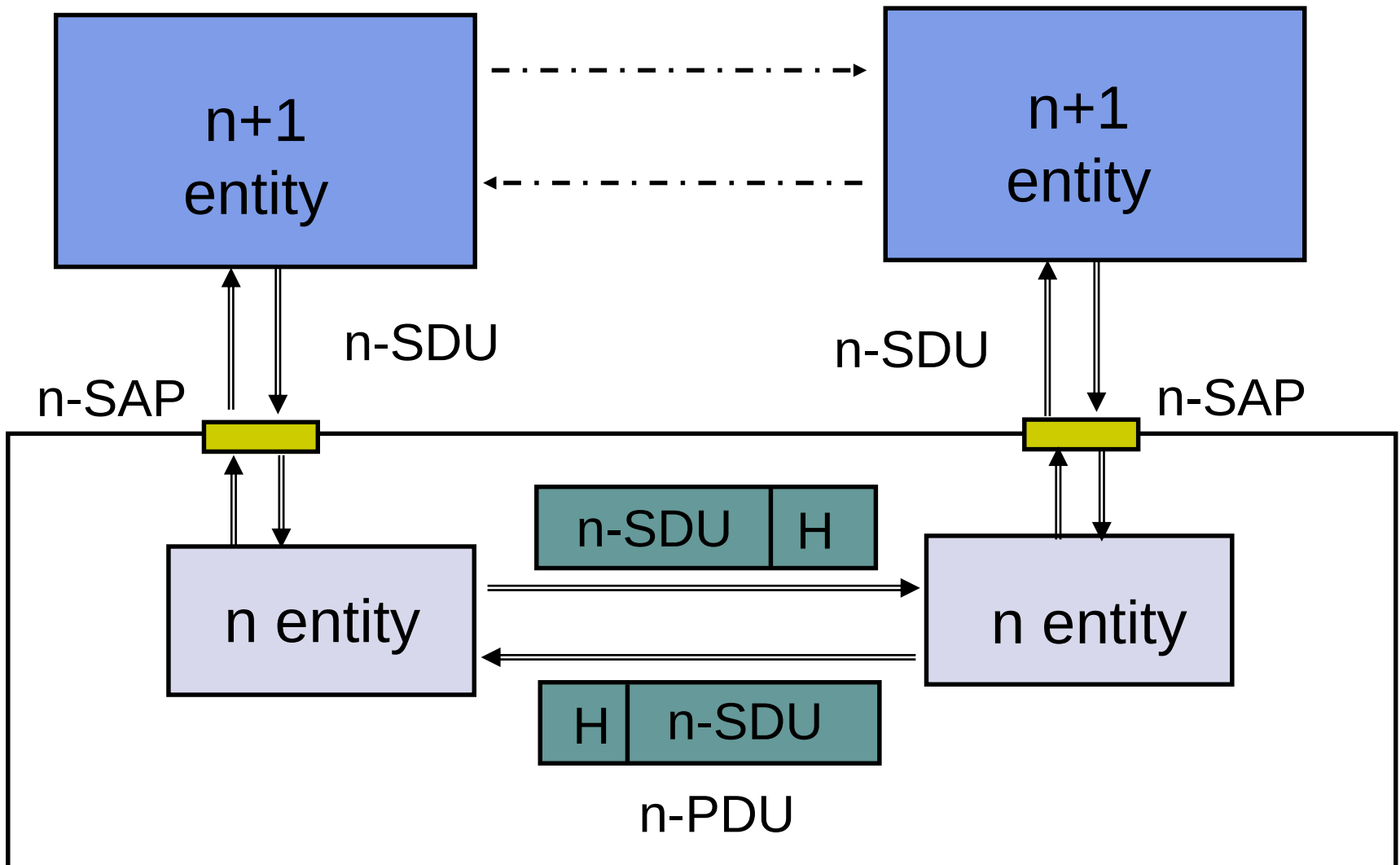
- Communication between peer processes is virtual and actually **indirect**
- Layer $n+1$ transfers information by invoking the services provided by layer n
 - Layer n serves layer $n+1$
 - Layer $n+1$ is a user of service provided by layer n
- Pass info from layer $n+1$ to layer n through SW called *Service Access Points (SAP's)*
- Layer $n+1$ is only interested in correct execution to transfer its PDU to the peer process (how execution is done is irrelevant)

OSI Unified View: Services (2)



- Each layer passes data & control information to the layer below it until the physical layer is reached and transfer occurs
- The data passed to the layer below is called a *Service Data Unit* (SDU)
- SDU's are *encapsulated* in PDU's + control info (i.e. what to do with data)

Layers, Services & Protocols



Connectionless & Connection-Oriented Services

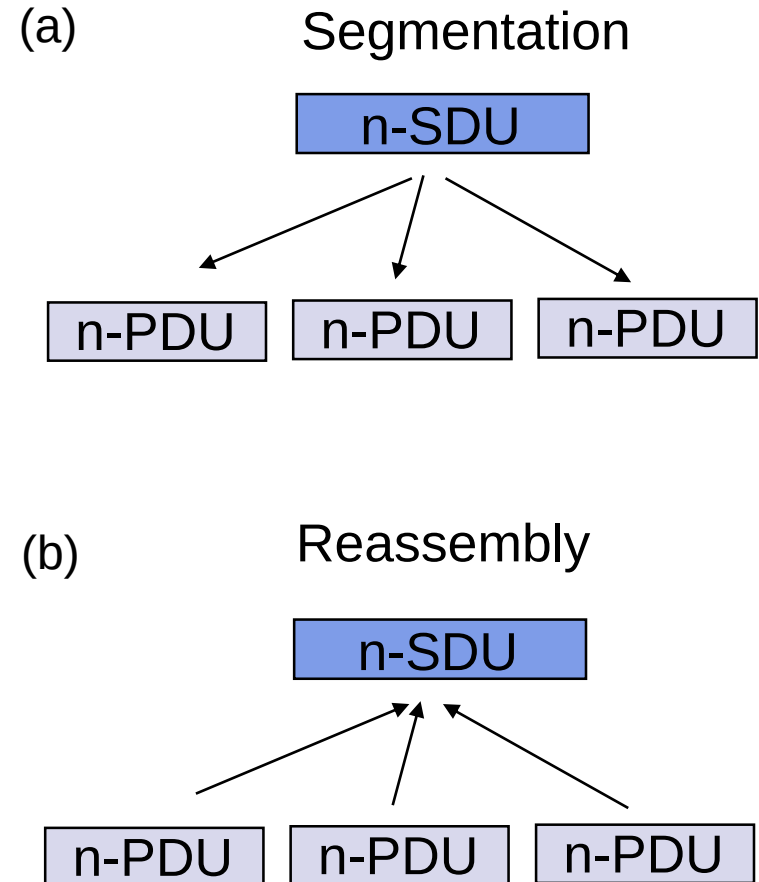


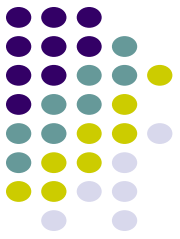
- Connection-Oriented
 - Three-phases:
 1. Connection setup between two SAPs to initialize state information
 2. SDU transfer
 3. Connection release
 - E.g. TCP, ATM (like a phone call)
- Connectionless
 - Immediate SDU transfer (control info contains address of destination)
 - No connection setup
 - E.g. UDP, IP
- Layered services not always of same type
 - TCP operates over IP
 - IP operates over ATM (like TV broadcast)

Segmentation & Reassembly



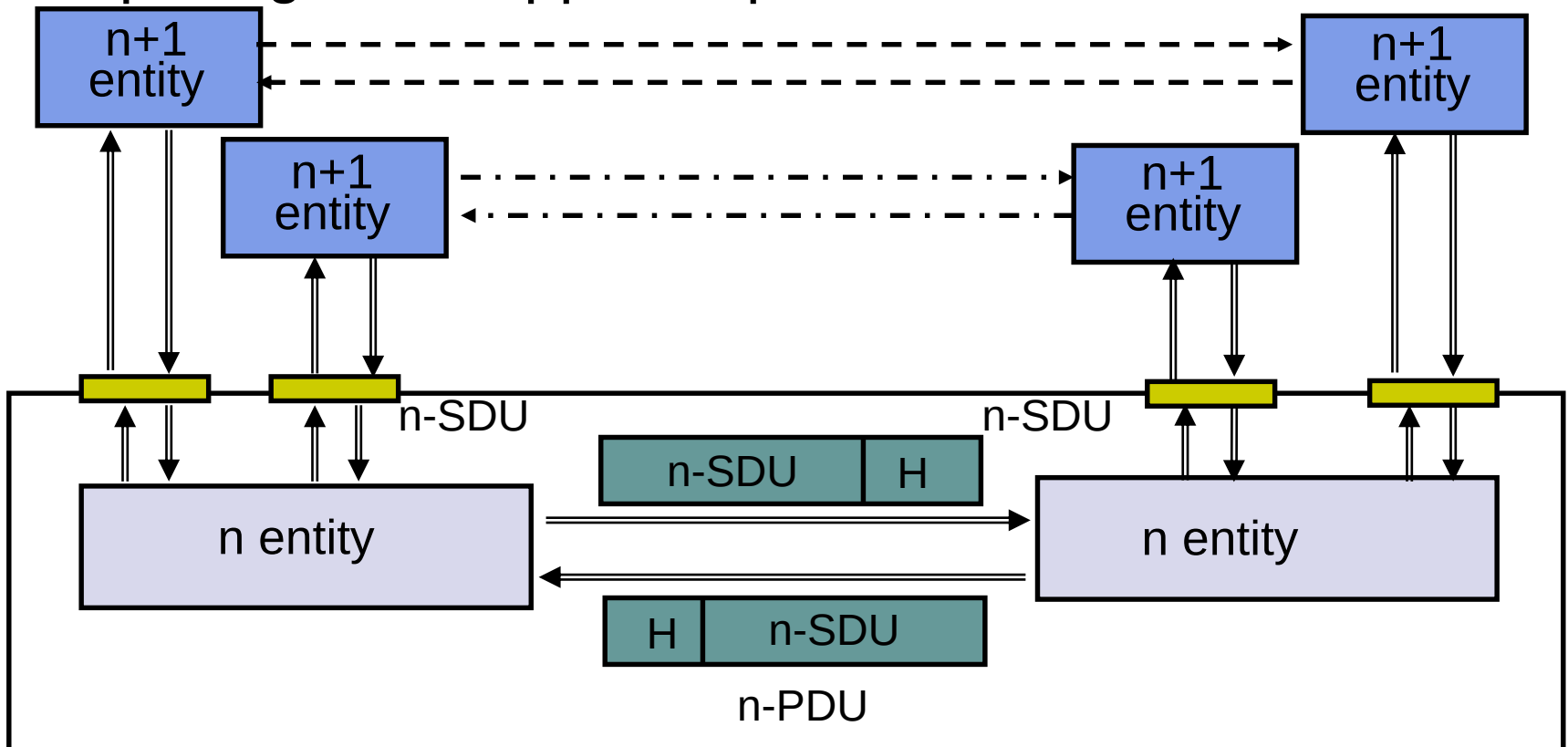
- A layer may impose a limit on the size of a data block that it can transfer for implementation or other reasons
- Thus a layer-n SDU may be too large to be handled as a single unit by layer-(n-1)
- Sender side: SDU is segmented into multiple PDUs
- Receiver side: SDU is reassembled from sequence of PDUs





Multiplexing

- Sharing of layer n service by *multiple* layer n+1 users
- Multiplexing tag or ID required in each PDU to determine which users an SDU belongs to
- Splitting is the opposite process





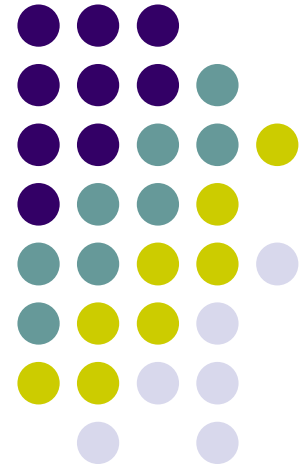
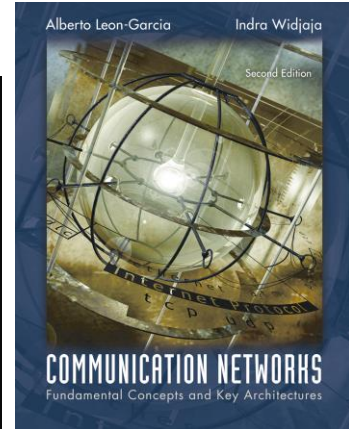
Summary

- Layers: related communications functions
 - Application Layer: HTTP, DNS
 - Transport Layer: TCP, UDP
 - Network Layer: IP
- Services: a protocol provides a communications service to the layer above
 - TCP provides connection-oriented reliable byte transfer service
 - UDP provides best-effort datagram service
- Each layer builds on services of lower layers
 - HTTP builds on top of TCP
 - DNS builds on top of UDP
 - TCP and UDP build on top of IP

Chapter 2

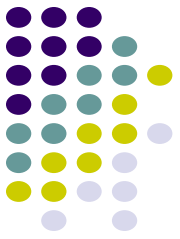
Applications and Layered Architectures

Example: TCP/IP Architecture
Widely Used Networking Technology

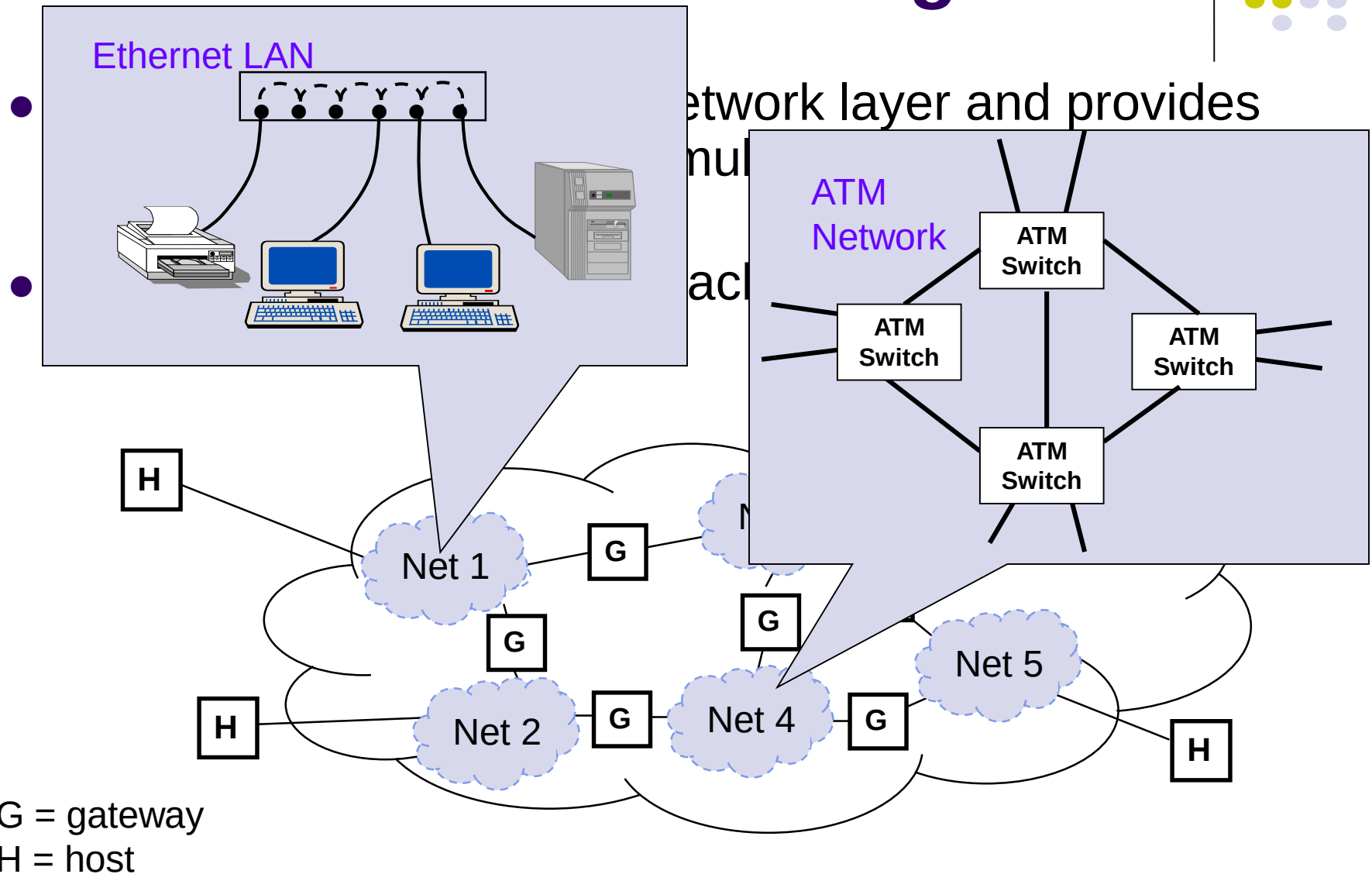




- TCP/IP net Architecture: set of protocols that allows comm. across multiple diverse nets
- Evolved out of military research to connect 3 nets
 1. ARPANET
 2. Packet radio net
 3. Packet satellite net
- Emphasis on robustness and flexibility
- Eventually led to Internet which connects world computers



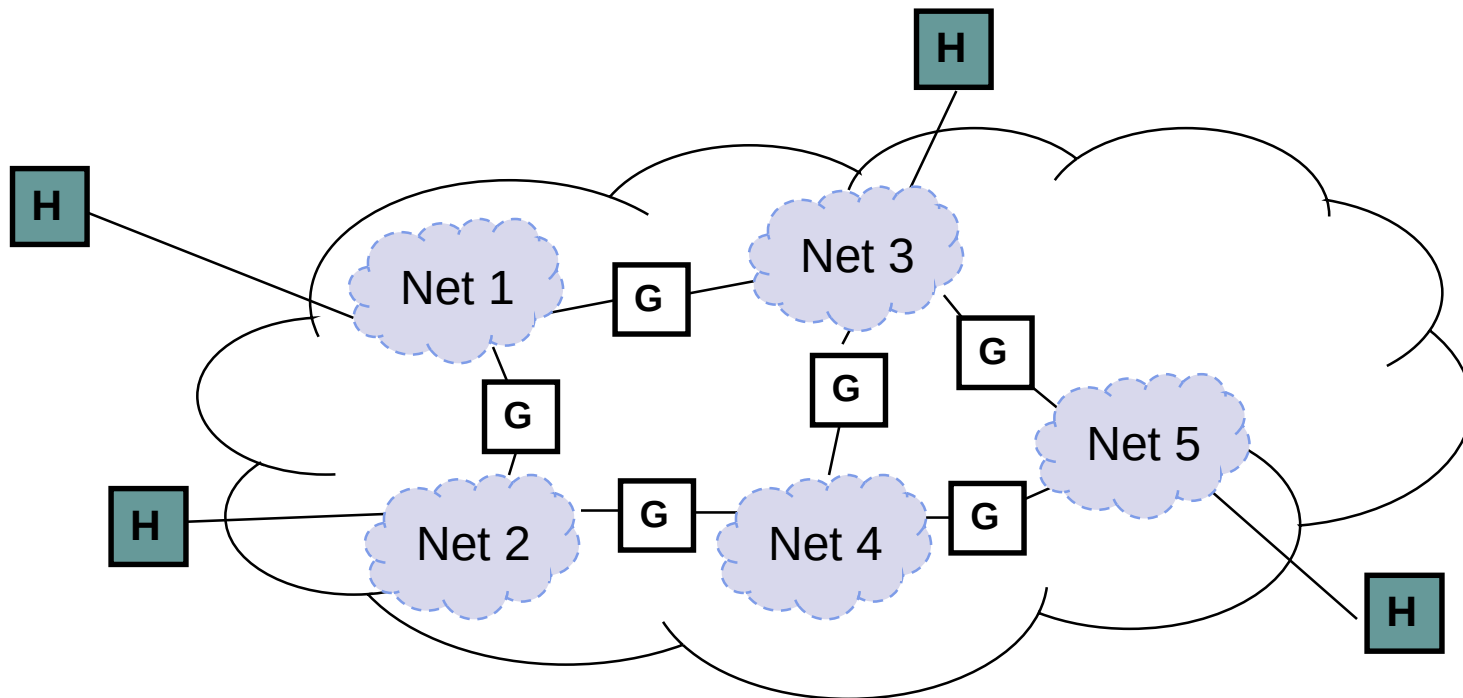
Internet: Internetworking



Why Internetworking?



- To build a “network of networks” or Internet
 - operating over multiple (different) network technologies
 - providing worldwide connectivity through IP packet transfer
 - independent of underlying network technologies
 - providing common interface to user applications



Application Layer



- TCP/IP Architecture consists of 4 layers
- TCP/IP application layer incorporates the functions of the top 3 OSI layers
- Application layer runs directly over transport layer

Transport Layer

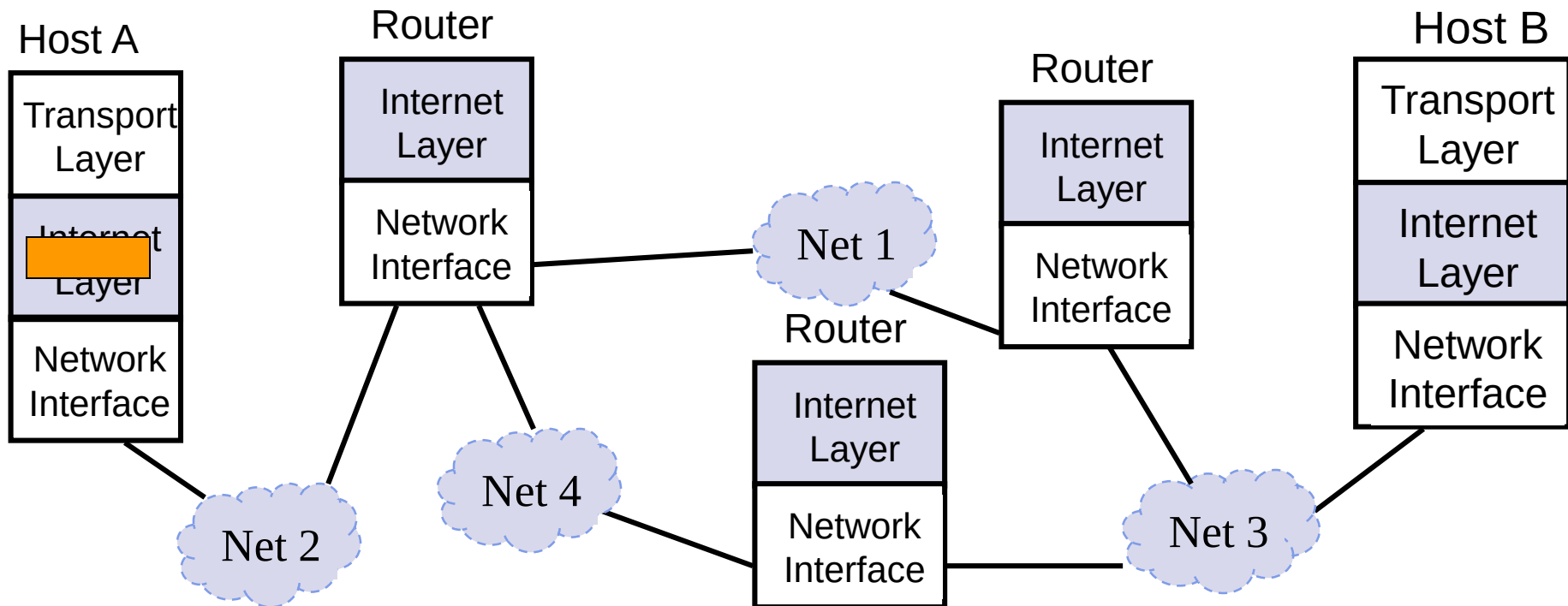


- Application layer runs over transport layer
- Offers two kinds of services
- TCP: connection oriented protocol for reliable transmission of a byte *stream*
- UDP (User Datagram protocol): Best-effort transfer of *individual* messages
- TCP/IP Arch. Does not require strict layering
(can bypass intermediate layers)

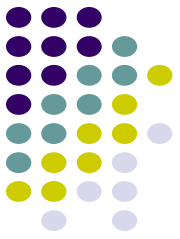


Internet Protocol Approach

- IP packets transfer information across Internet
Host A IP → router → router... → router → Host B IP
- IP layer in each router determines next hop (router)
- Network interfaces transfer IP packets across networks

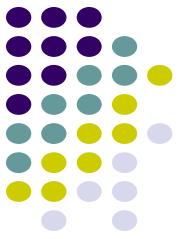


Internet Protocol Approach



- Internet layer corresponds to the net. Layer in OSI
- Key requirement is globally unique address for machines attached to internet
- IP packets exchanged between routers without connection setup
- Packets routed independently (so possibility for different paths); no connection set up

Internet Protocol Approach



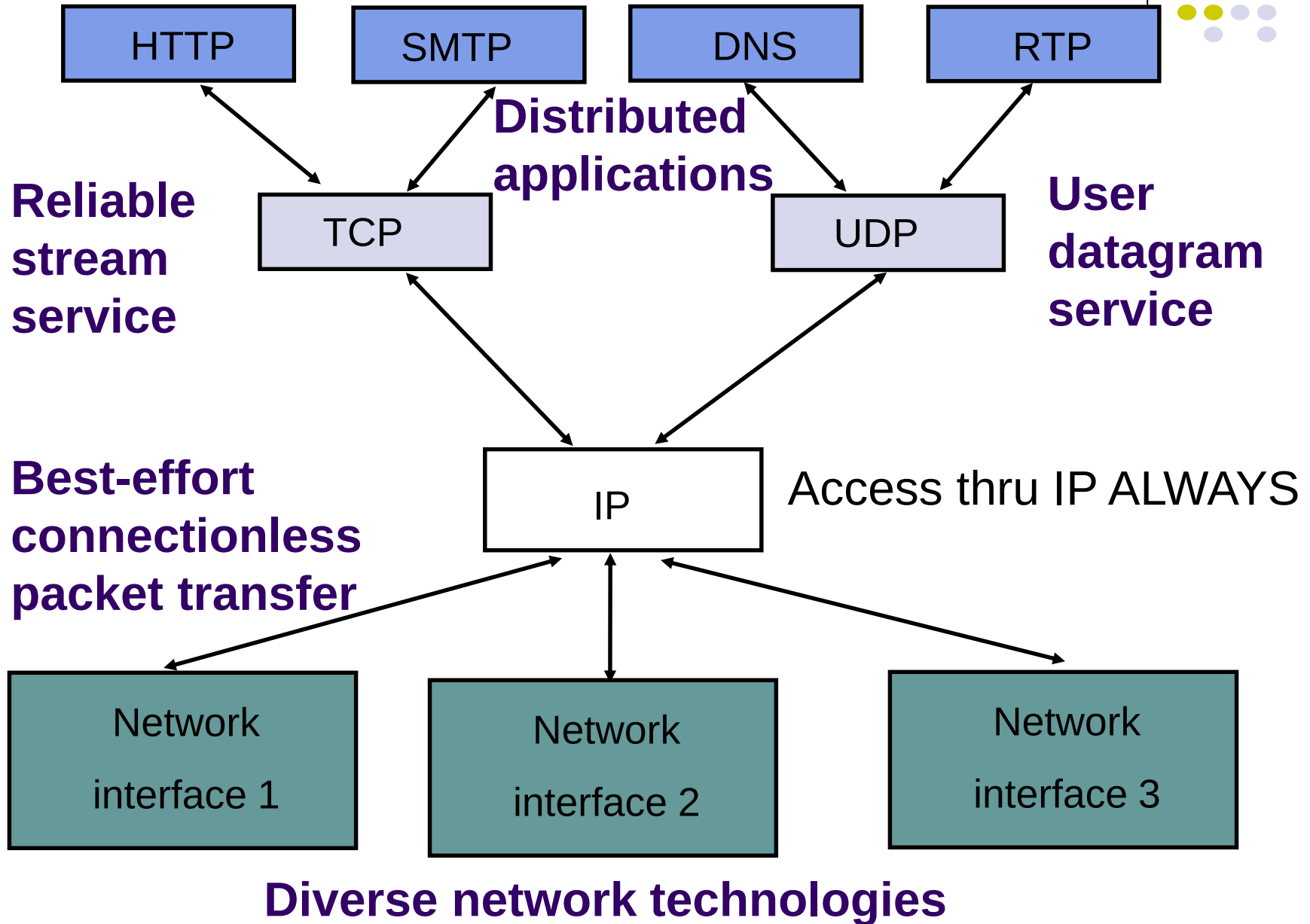
- Connectionless approach makes system robust
 - if failure occurs, packets routed around failure points;
 - no need to setup connections again
- Gateways can discard packets
- Responsibility of recovering losses passed to transport layer

Network Interface Layer

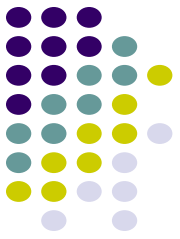


- Deals with network specific aspects for transfer of packets
- Various networks implies the need for various interfaces (ATM, Ethernet, Token ring...)
- Protocols that access intermediate networks: encapsulates packet IP packet into packet or frame of underlying network
- Network interface is technology dependent
- Internet layer is technology independent (transparent to underlying network, i.e. does not depend on details of underlying networks)

TCP/IP Protocol Suite



Hourglass shape of TCP/IP

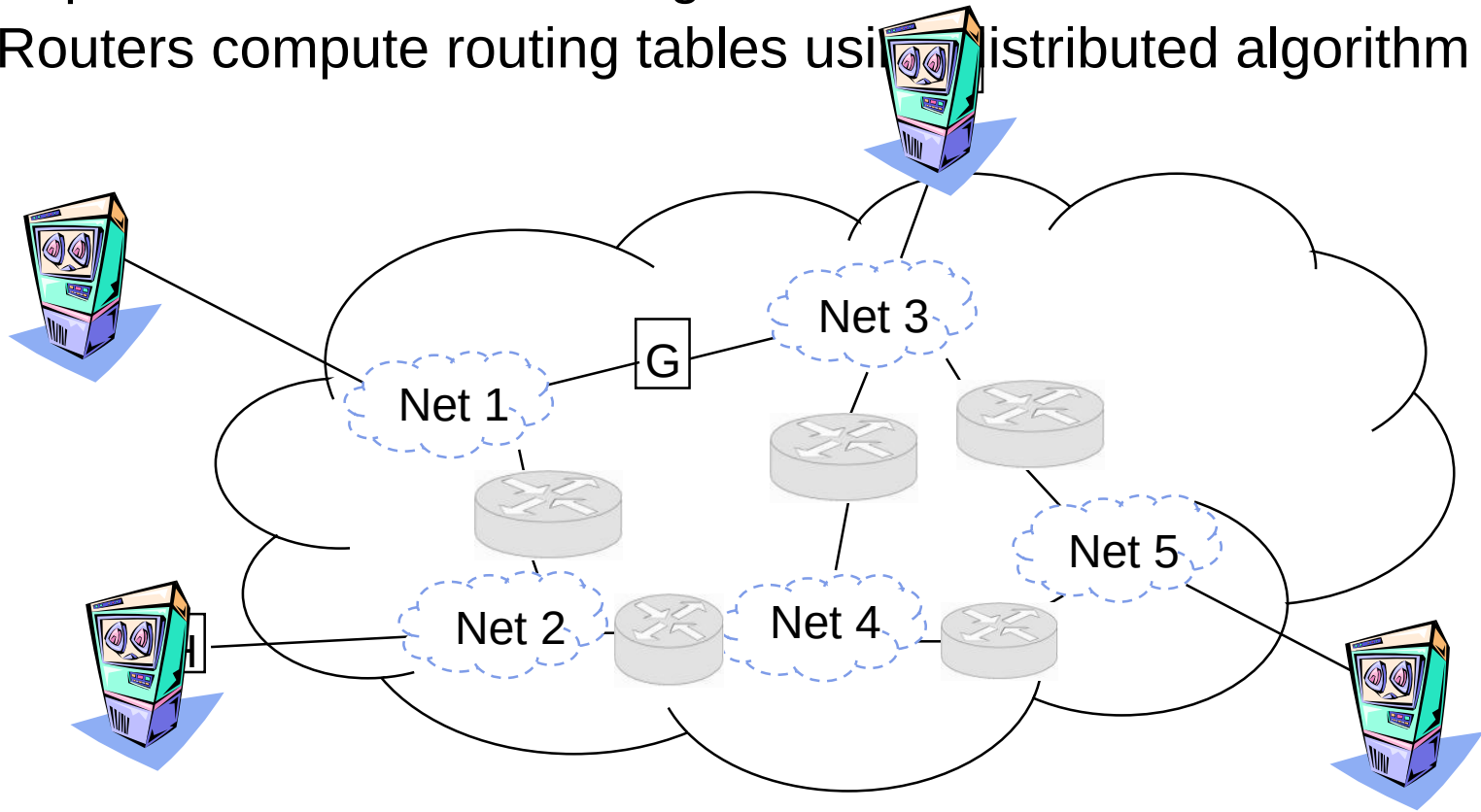


- Why TCP/IP is so powerful?
- Single IP protocol over various nets provides indep. from underlying technologies
- Communication services of TCP and UDP provide network indep. platform on which applications can be developed.
- So multiple technologies can co-exist

Addressing & Routing



- IP address divided into 2 parts: Net ID + Host ID
- Hierarchical address: Net ID + Host ID
- Net ID obtained from Org. authorized to give out addresses
- IP packets routed according to Net ID
- Routers compute routing tables using distributed algorithm





- Each host is identified by a globally unique IP address
- Identifies host network interface (rather than host)
- Router: a node connected to 2 or more physical networks; with each net interface assigned to unique IP address

Names and IP Addresses



- Routing is done based on 32-bit IP addresses
- Dotted-decimal notation
 - 128.100.11.1
- Hosts are also identified by name
 - Easier to remember
 - Hierarchical name structure
 - tesla.comm.utoronto.edu
- Domain Name System (DNS) provides conversion between names and addresses

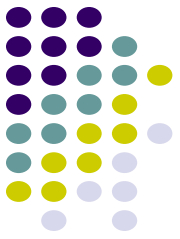
Physical Addresses



- LANs assign **physical address** to the equipment attached to the network
- Format of physical address depends on particular type of network used
- Example: Ethernet uses 48-bit addresses
 - Each Ethernet network interface card (NIC) has globally unique Medium Access Control (MAC) or physical address
 - First 24 bits identify NIC manufacturer; second 24 bits are serial number
 - This guarantees a unique phys address to each machine in the LAN net
 - 00:90:27:96:68:07 12 hex numbers

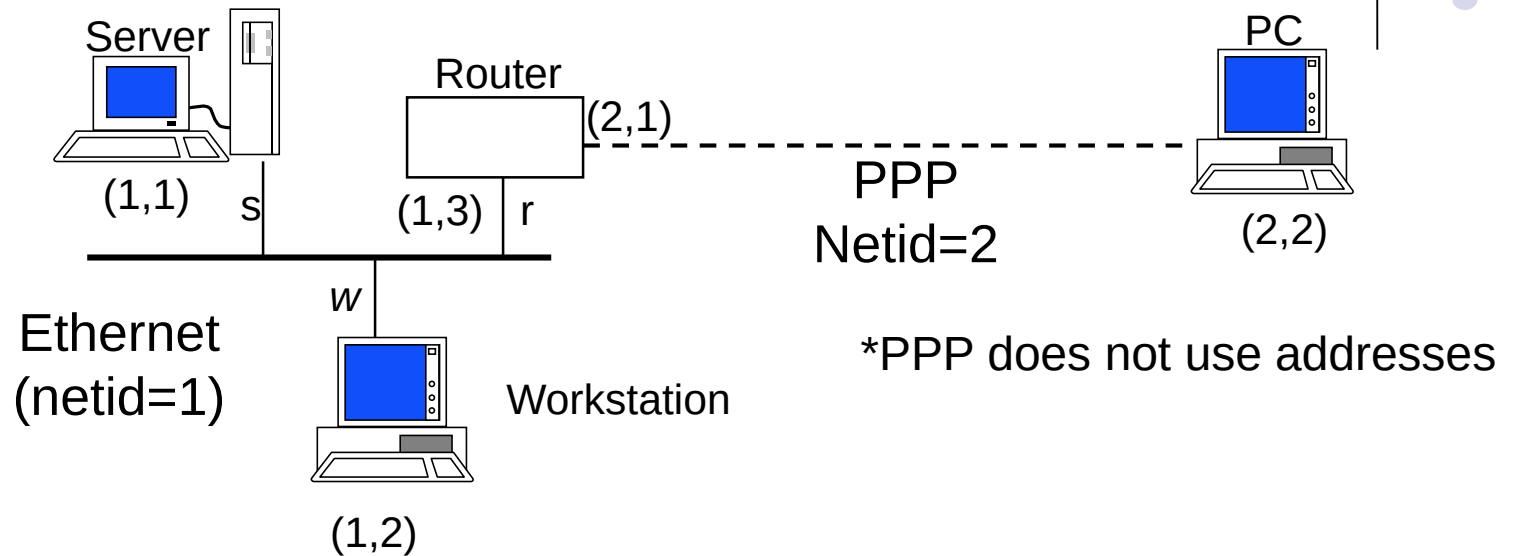


Intel



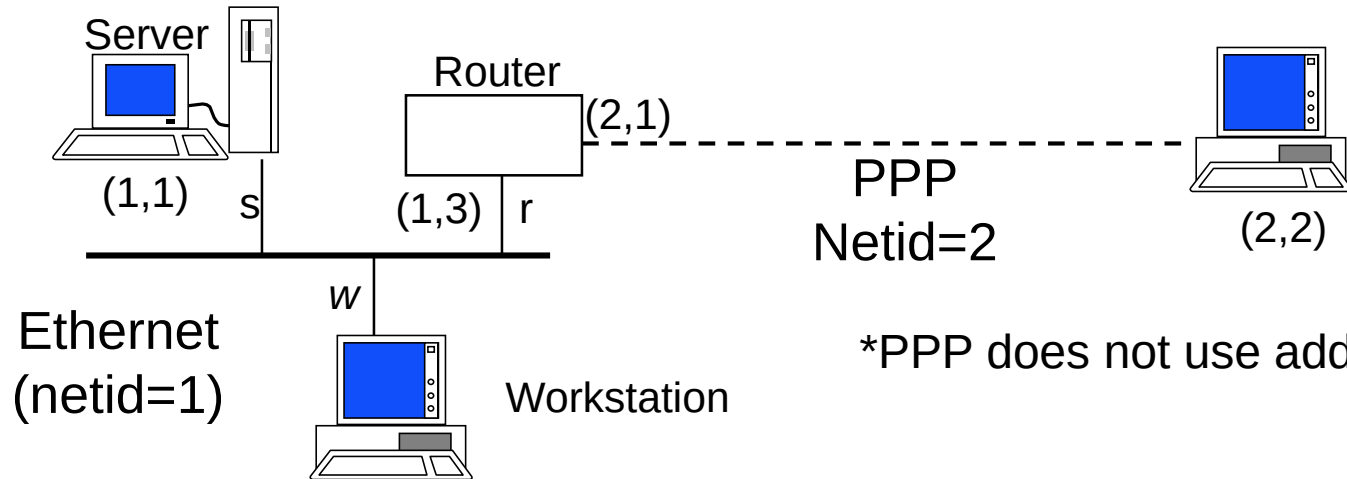
- The network uses its own **logical address** to transfer packets or frames to the appropriate destination
- IP address needs to be matched to physical address at each IP network interface (e.g., using ARP)

Example: How IP datagrams is sent across internet



	netid	hostid	Physical address
server	1	1	s
workstation	1	2	w
router	1	3	r
router	2	1	-
PC	2	2	-

Sending packet within same net



*PPP does not use addresses

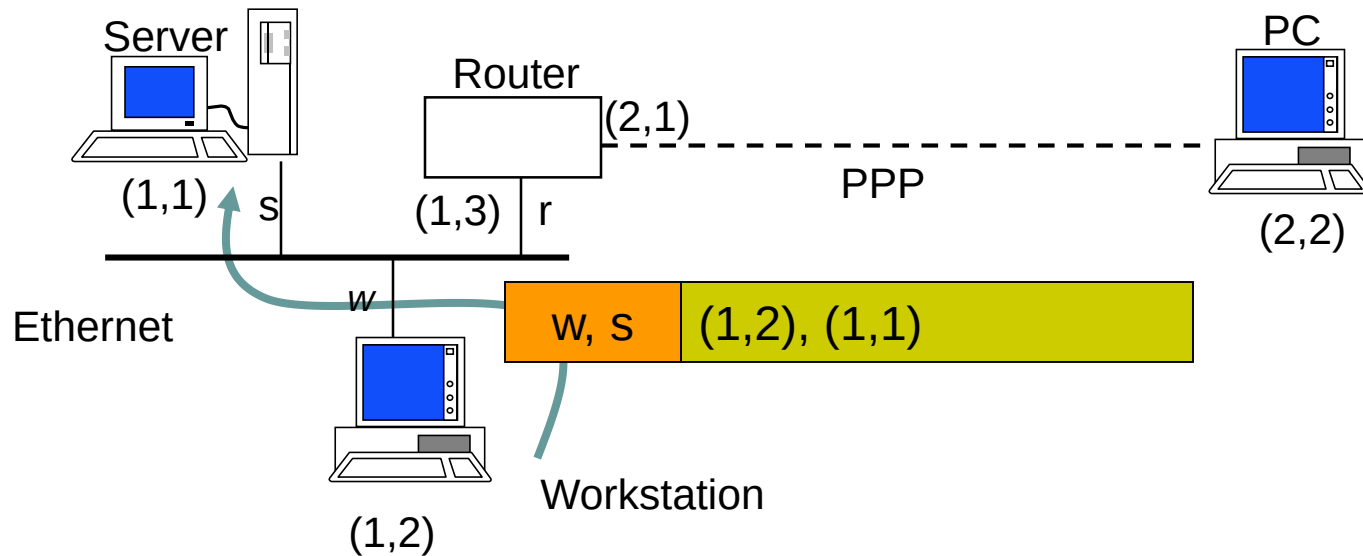
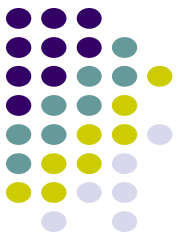
- Workstation wants to send IP datagram to server
- IP Datagram has W IP address & S IP address in packet header
- IP routing table at W indicates $(1,1)$ connected to **same** network, so IP packet is encapsulated in Ethernet frame with addresses w and s
- Figures out that S connected to same net (with phys. address s)
- IP datagram passed to Ethernet device driver which prepares an Ethernet frame

w, s	$(1,2), (1,1)$
------	----------------



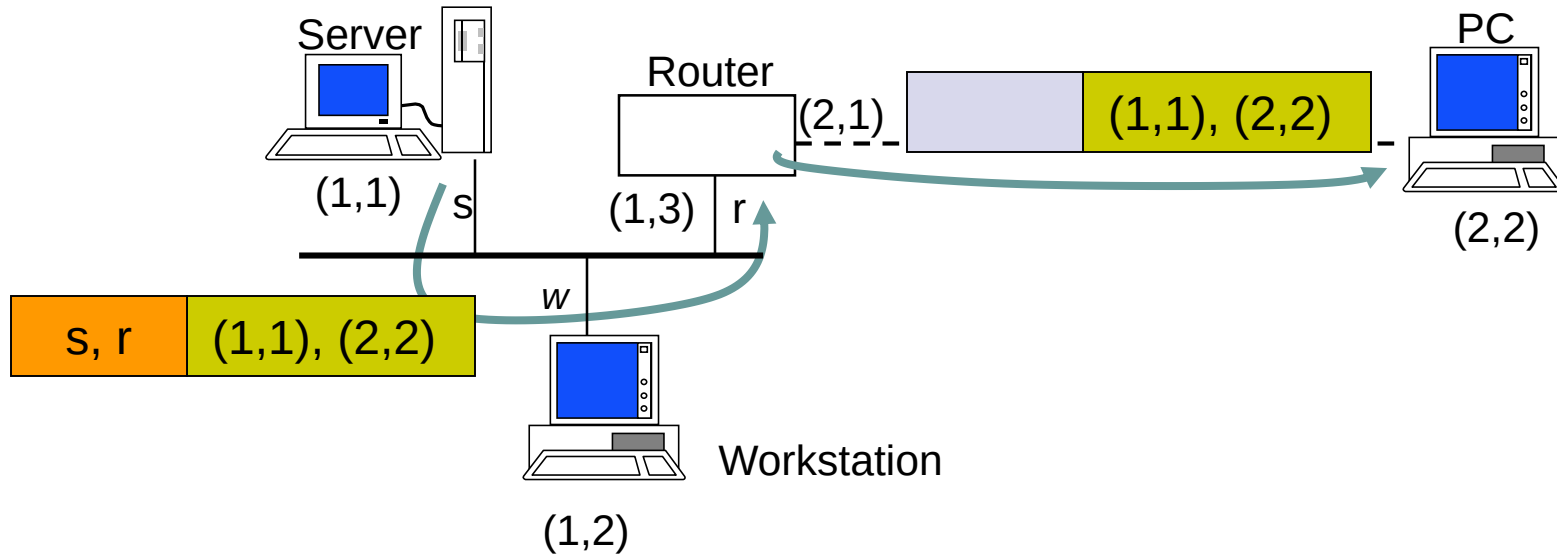
- Header in new frame contains:
 1. source **phys.** Address **w** and destination **phys.** address **s**
 2. Type Field set to value corresponding to IP (why we need this?)
- Ethernet frame broadcast over LAN (why broadcast)
- Server's NIC recognizes frame is intended for it and captures it
- NIC find Type Field=IP and so passes IP datagram up to IP layer

Sending IP packet between two nets



1. IP packet has (1,2) IP address for source and (1,1) IP address for destination
2. IP table at workstation indicates (1,1) connected to same network, so IP packet is encapsulated in Ethernet frame with addresses w and s
3. Ethernet frame is broadcast by workstation NIC and captured by server NIC
4. NIC examines protocol type field and then delivers packet to its IP layer

IP packet from server to PC

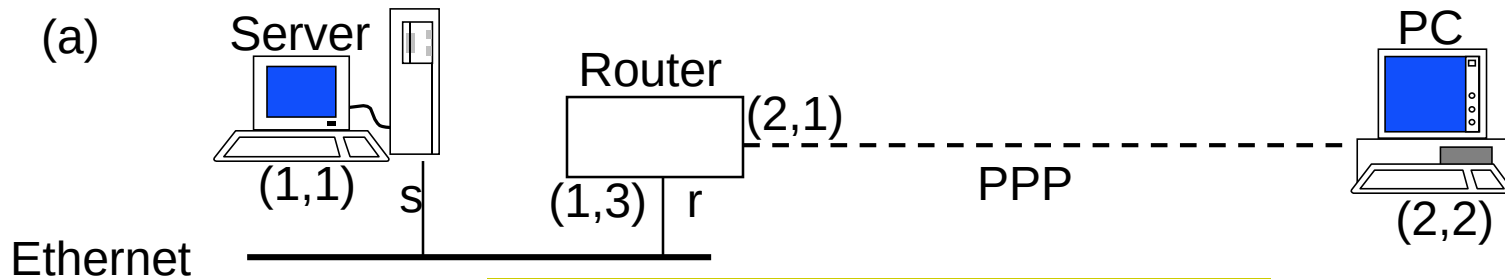


1. IP packet has (1,1) and (2,2) as IP source and destination addresses
2. Server checks routing table to see if it has IP address of PC
3. Server checks to see if routing table has entry with the same network id portion of IP address of PC
4. IP table at server indicates packet should be sent to router (default), so IP packet is encapsulated in Ethernet frame with addresses s and r
5. Ethernet frame is broadcast by server NIC and captured by router NIC
6. NIC of router examines protocol type field and then delivers packet to its IP layer



1. IP layer examines IP packet destination address and determines IP packet should be routed to (2,2) (not to router itself)
2. Router's table indicates (2,2) is directly connected via PPP link
3. IP packet is encapsulated in PPP frame and delivered to PC
4. PPP at PC examines protocol type field and delivers packet to PC IP layer

How the layers work together

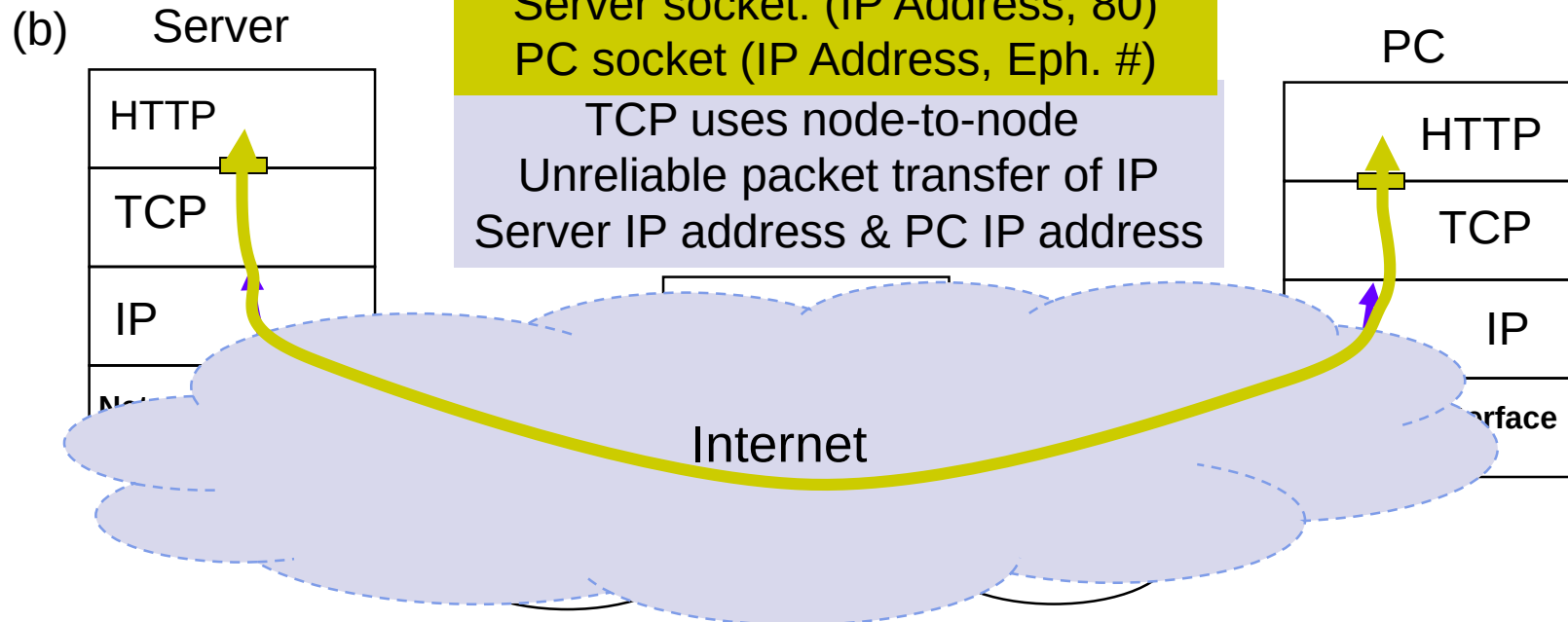


HTTP uses process-to-process
Reliable byte stream transfer of
TCP connection:

Server socket: (IP Address, 80)

PC socket (IP Address, Eph. #)

TCP uses node-to-node
Unreliable packet transfer of IP
Server IP address & PC IP address





- Click on a link at PC
- TCP connection established between PC and Server
- GET request is issued by HTTP application
- Passed to TCP layer
- TCP encapsulates request in TCP segment:
 - Header contains port no of client HTTP process and port no. for server HTTP process



- TCP segment passed to IP layer
- IP layer encapsulates TCP segment into IP packet
 - header contains IP addresses of sender and destination
 - Header contains protocol field (indicating layer above is TCP)
- IP packets encapsulated using PPP and sent to router
- Router encapsulates IP datagram for server in Ethernet frame